

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Фронт-энд разработка

ОТЧЁТ

Лабораторная работа №2
Работа с API и асинхронными запросами

Выполнил:
студент группы J3212
Мирасов Константин

Проверил:
Добряков Д. И.

Санкт-Петербург
2026 г.

Задача

Подключить сайт AxonHub к HTTP API. Реализовать асинхронную загрузку ресурсов, регистрацию и вход, сохранение действий пользователя, а также понятные состояния загрузки и ошибок.

Ход работы

1. Клиент и API

HTML-клиент работает на порту 8080, локальный JSON Server — на порту 3001. Начальные данные находятся в `mock/seed.json`; при первом запуске создаётся рабочая база `db.json`. Доступ к операциям ограничивают обработчики `access.cjs` и `middleware авторизации`.

Каталог получает JSON-массив ресурсов и создаёт строки DOM. Фильтры применяются к загруженным данным, а их значения сохраняются в URL.

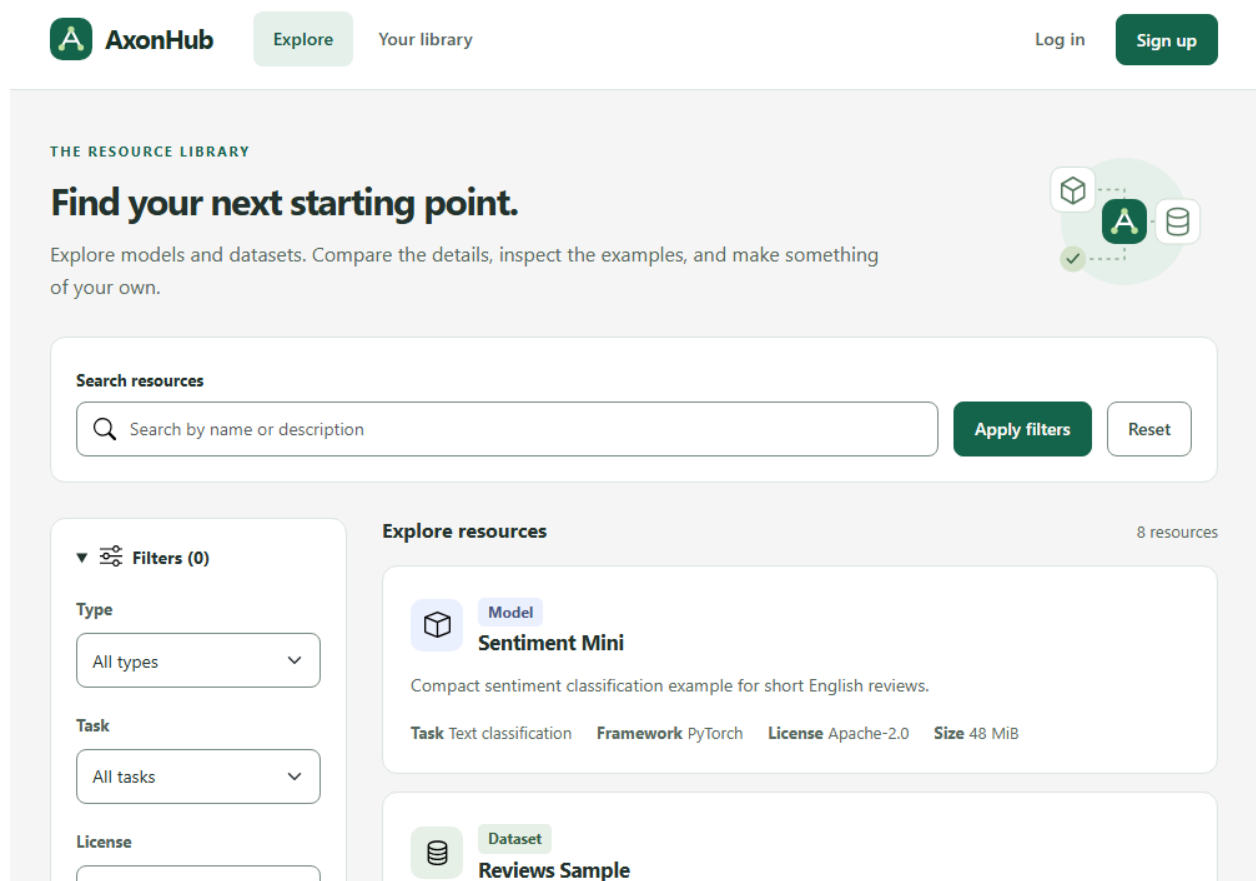


Рисунок 1. Каталог ресурсов, загруженных из API

2. Асинхронные запросы

HTTP-запросы вынесены в `assets/js/services`. Общая функция добавляет JSON-тело и заголовок авторизации, ограничивает ожидание и преобразует неуспешный статус в ошибку. Ответ 401 сбрасывает недействительную сессию.

`assets/js/services/http.js` · строки 12–33

```
if (authenticated && getSession()) {
  headers.Authorization = `Bearer ${getSession().accessToken}`;
}

const response = await fetch(`${apiBase}${path}`, {
  method,
  headers,
  body: body ? JSON.stringify(body) : undefined,
  signal: AbortSignal.timeout(10000),
});

if (!response.ok) {
  if (response.status === 401) {
    clearSession();
  }

  const error = new Error(`Request failed (${response.status}).`);

  error.status = response.status;

  throw error;
}
```

Страница модели запрашивает ресурс по ID. Полученные данные проходят проверку структуры и передаются функциям отображения; обсуждение и социальные действия загружаются отдельно.

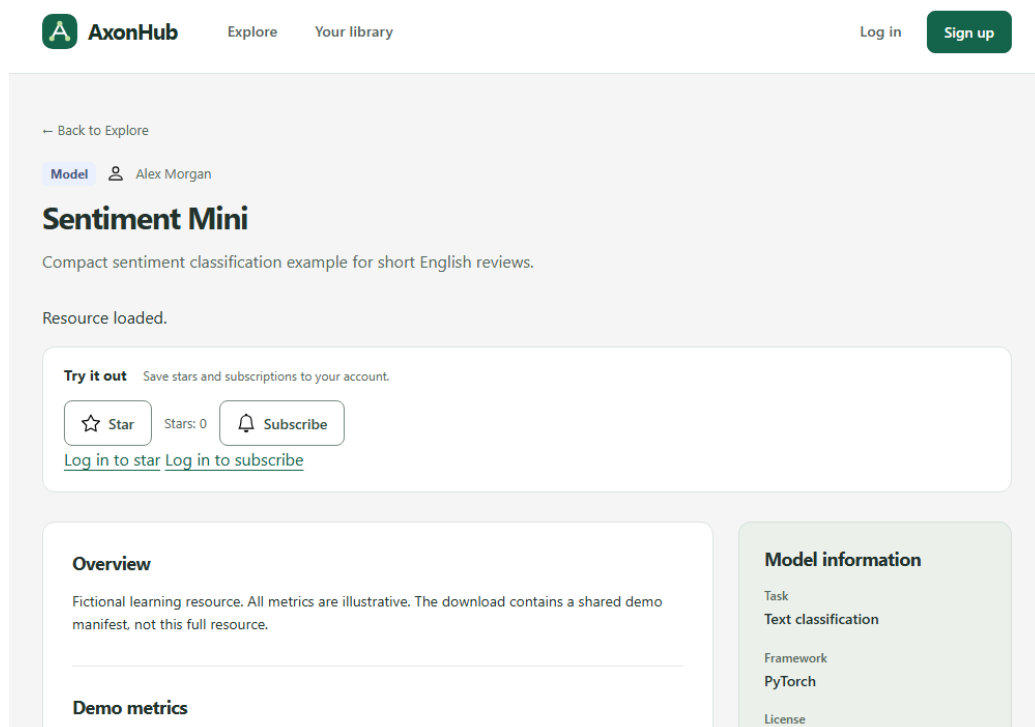


Рисунок 2. Страница модели, сформированная по данным сервера

3. Регистрация, вход и личный кабинет

Регистрация отправляет email, пароль и отображаемое имя. После успешного ответа пользователь входит в систему и попадает в кабинет. Новый аккаунт начинает работу с пустой библиотекой.

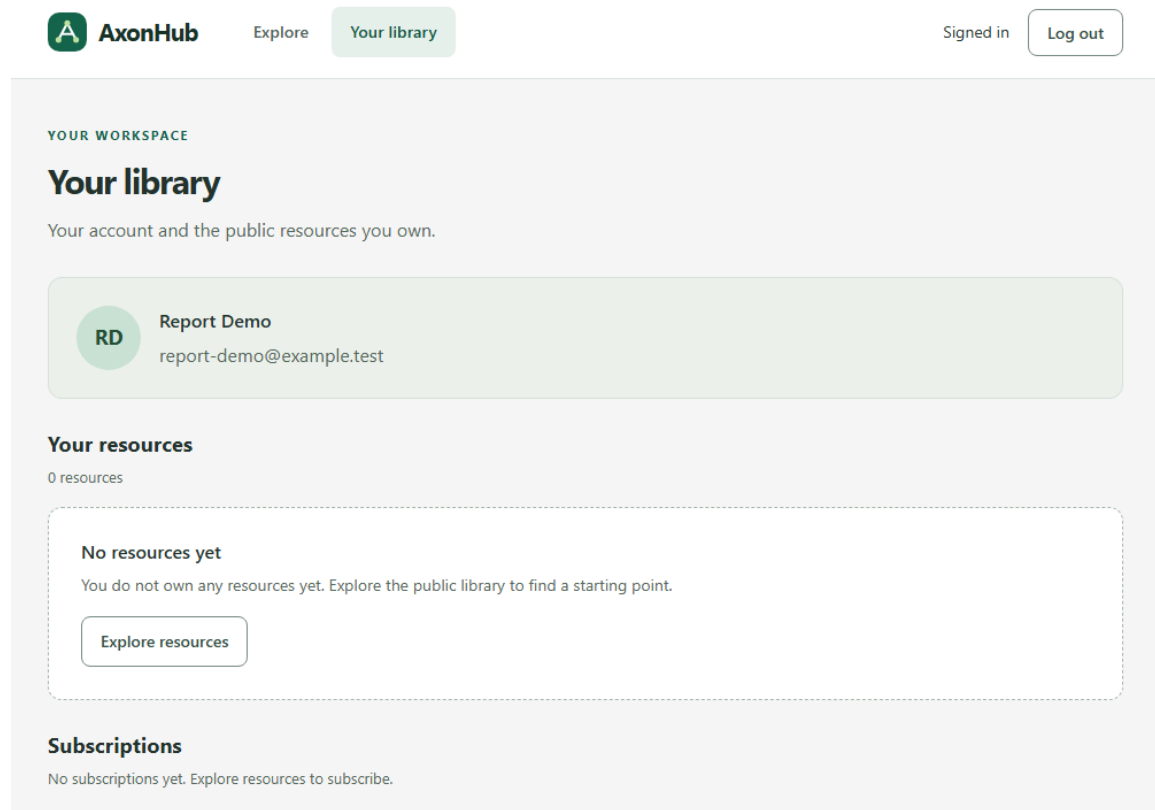


Рисунок 3. Кабинет нового пользователя

В sessionStorage записываются токен и разрешённые поля пользователя: id, email, displayName. Пароль в браузерное хранилище не сохраняется. Выход очищает сессию. Гостевые действия предлагают войти в аккаунт.

assets/js/forms.js · строки 48–60

```
const email = form.elements.email.value.trim().toLowerCase();
const password = form.elements.password.value;
const session = isRegistering
  ? await register(email, password, form.elements.displayName.value)
  : await login(email, password);

accountCreated = isRegistering;
saveSession(session);
location.assign(
  isRegistering
    ? 'profile.html'
    : getReturnPath(new URLSearchParams(location.search).get('returnTo')),
);
```

4. Звёзды, подписки и комментарии

Звёзды и подписки записываются в API и сохраняются после перезагрузки. Подписка добавляет ресурс в библиотеку пользователя. Удалять связи и редактировать комментарии может их владелец.

Комментарии содержат обычный текст длиной 1–1000 символов. HTML-подобная строка отображается текстом. Для удаления предусмотрено подтверждение; при ошибке отправки или редактирования черновик остаётся в форме.

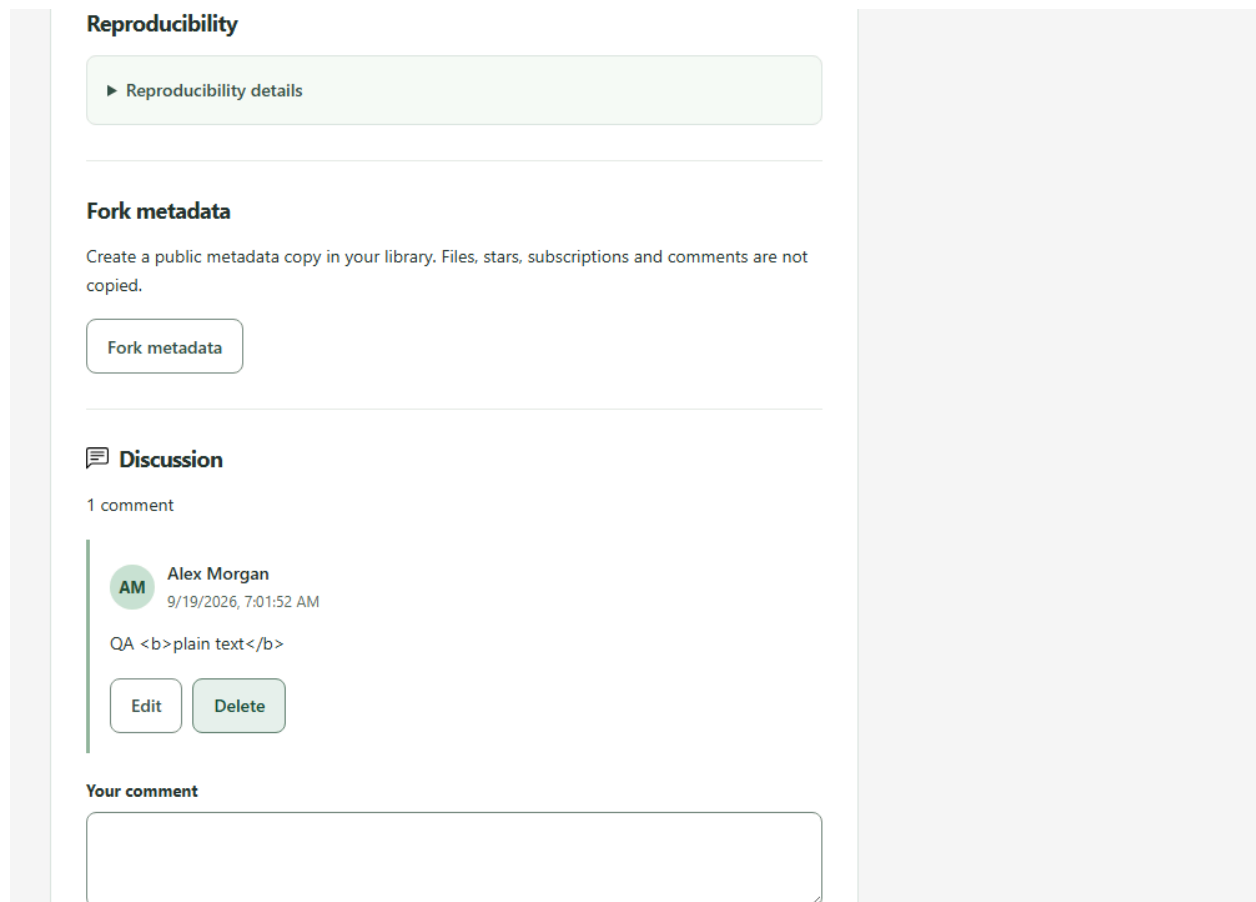


Рисунок 4. Обсуждение ресурса с добавленным комментарием
assets/js/services/social.js · строки 3–12

```
export function editComment(id, body) {
  return requestJson(`/discussions/${id}`, {
    method: 'PATCH',
    authenticated: true,
    body: { body, updatedAt: new Date().toISOString() },
  });
}

export function deleteComment(id) {
  return requestJson(`/discussions/${id}`, { method: 'DELETE', authenticated: true });
}
```

5. Копирование метаданных и ошибки

Fork metadata создаёт ресурс с новым ID и владельцем. Описательные поля копируются, `sourceResourceId` указывает на оригинал, счётчик загрузок равен нулю. Файлы, звёзды и обсуждения не копируются; повторное действие открывает существующую копию пользователя.

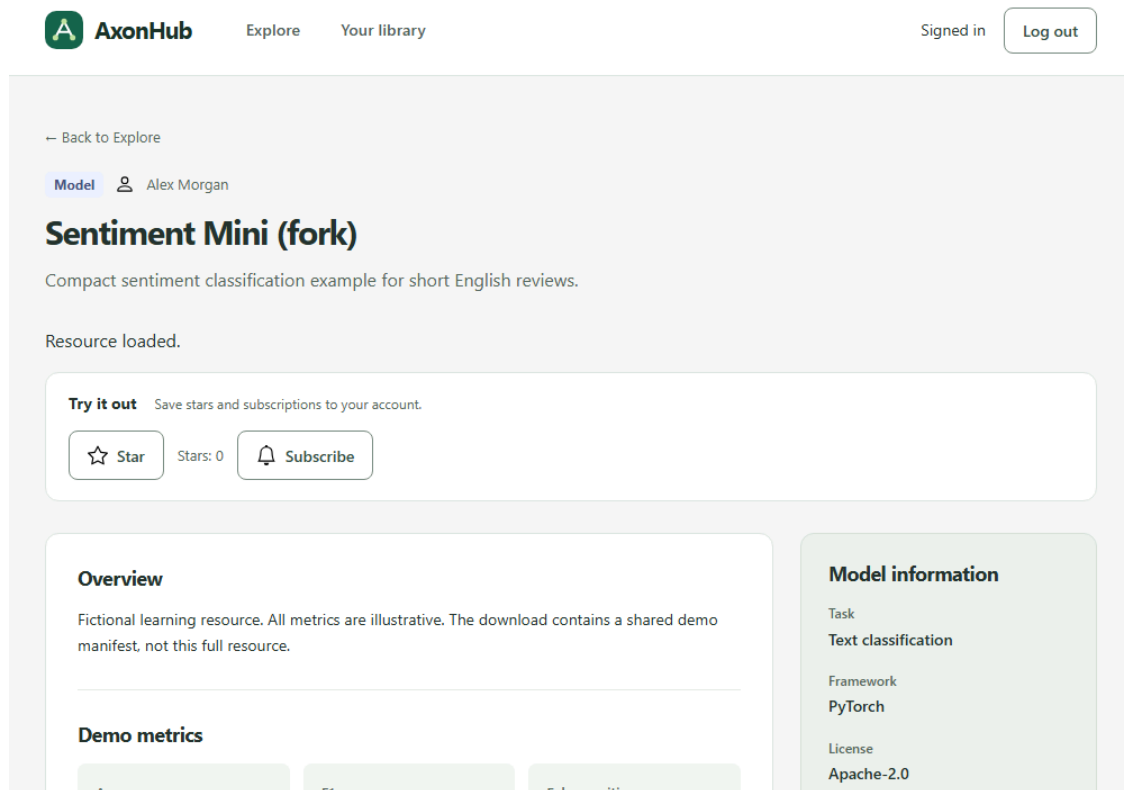


Рисунок 5. Копия метаданных ресурса

При недоступности API интерфейс показывает сообщение и кнопку `Retry`. Неверный идентификатор и отсутствующий ресурс имеют отдельные сообщения. Во время запроса кнопки блокируются от повторной отправки, после завершения снова становятся доступны.

Вывод

AxonHub получает данные через HTTP API и сохраняет пользовательские действия в базе. Сервисы запросов отделены от отображения страниц, сессия определяет доступные действия, а ошибки имеют понятные сообщения и возможность повторной загрузки.